

Laporan Tugas Kecil 2 IF2211 Strategi Algoritma

Voxelization Objek 3D menggunakan Octree

13524053@std.stei.itb.ac.id
mharisputras.work@gmail.com

13524110@std.stei.itb.ac.id
jenniferkhang07@gmail.com



Disusun oleh:

Muhammad Haris Putra Sulastianto 13524053

Jennifer Khang 13524110

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung**

2026

Daftar Isi

1	Permasalahan	3
2	Teori Singkat	4
2.1	Voxelization	4
2.2	Octree	4
2.3	Divide and Conquer	5
2.4	Bounding Box Collision Detection	5
2.4.1	Axis-Aligned Bounding Box	6
2.4.2	Hubungan Octree dengan AABB	6
3	Pendekatan Solusi	7
4	Implementasi	7
4.1	Struktur Data	7
4.1.1	Types	7
4.1.2	Mesh	8
4.1.3	Octree	8
4.2	Algoritma Voxelization	10
4.2.1	Inisialisasi	10
4.2.2	Divide and Conquer	10
4.2.3	Kondisi Berhenti	12
4.2.4	Pengecekan Interseksi	12
4.2.5	Output	12
4.3	Kompleksitas	13
5	Extras	13
5.1	<i>Concurrency</i>	13
5.2	Object Viewer	14
6	Uji Coba Program	15
6.1	line.obj	15
6.2	teapot.obj	16
6.3	cow.obj with maxDepth 6	17

6.4	pumpkin.obj	18
6.5	cow.obj with maxDepth 8	19
A	Lampiran	22
A.1	Pranala Ke Repositori	22
A.2	Pernyataan	22
A.3	Tabel Spesifikasi Tugas Kecil	23

1 Permasalahan

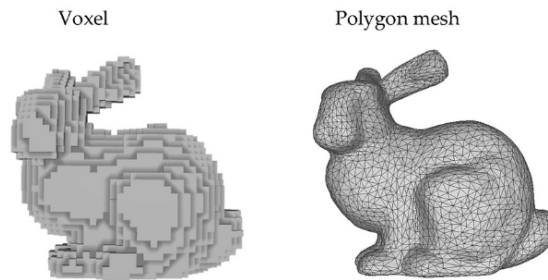


Figure 1: Visualisasi Voxelization [6]

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data Octree, yaitu sebuah tree yang setiap percabangan terdiri dari 8 anak. Melalui Octree, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.

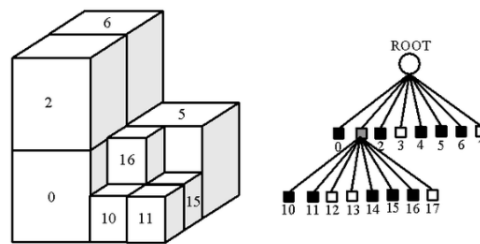


Figure 2: Visualisasi Octree terhadap 3D Model [4]

Pada tugas ini, tim diminta untuk membuat sebuah solusi yang dapat melakukan proses Voxelization secara efisien menggunakan algoritma Divide and Conquer melalui pembuatan Octree. Maka dari itu, algoritma ini akan menjadi kunci pusat dalam menyelesaikan pembagian ruang 3D menjadi suatu bagian permukaan objek yang dipertahankan dan direpresentasikan sebagai Voxel atau kubus. Selain itu, program memiliki format input dan output sendiri berupa file berformat '.obj' yang berisi informasi vertex dan face, serta menghasilkan keluaran berupa file '.obj' baru yang merepresentasikan objek dalam bentuk voxel.

2 Teori Singkat

2.1 Voxelization

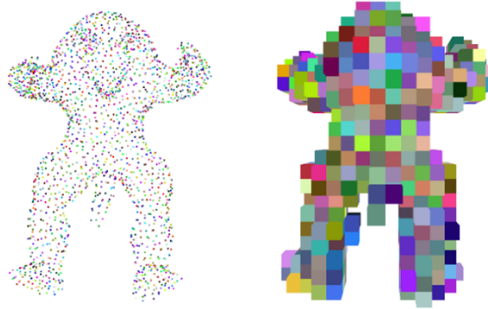


Figure 3: Voxelization from Point Cloud [3]

Point cloud dan triangle mesh adalah representasi geometri yang sangat fleksibel, tetapi bersifat tidak teratur (irregular). Sebagai alternatif, voxel grid merupakan tipe geometri 3D yang terdefinisi pada grid tiga dimensi yang teratur (regular), di mana sebuah voxel dapat dianggap sebagai padanan pixel dalam ruang 2D.

Voxelization adalah proses mengonversi representasi geometri yang tidak terstruktur (seperti point cloud atau mesh) menjadi representasi voxel grid yang terstruktur, sehingga memudahkan pemrosesan lebih lanjut seperti spatial analysis, collision detection, dan computational optimization.

2.2 Octree

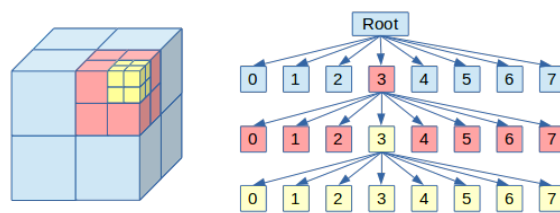


Figure 4: Octree Representation [2]

Octree adalah struktur data pohon di mana setiap node internal (non-leaf) memiliki delapan anak. Setiap node merepresentasikan suatu ruang tertentu dalam bentuk axis-aligned bounding box (AABB), serta memiliki titik tengah yang digunakan untuk membagi ruang menjadi delapan bagian melalui tiga bidang sejajar sumbu X, Y, dan Z. Setiap anak dari node tersebut merepresentasikan salah satu dari delapan subdivisi ruang hasil pembagian ini [1].

Node dalam octree dapat berupa:

1. Node internal, yang memiliki titik tengah sendiri dan delapan anak, dengan ruang yang lebih kecil dari parent-nya.
2. Leaf node, yang menyimpan data aktual (misalnya segitiga atau objek) yang ingin diproses.

2.3 Divide and Conquer

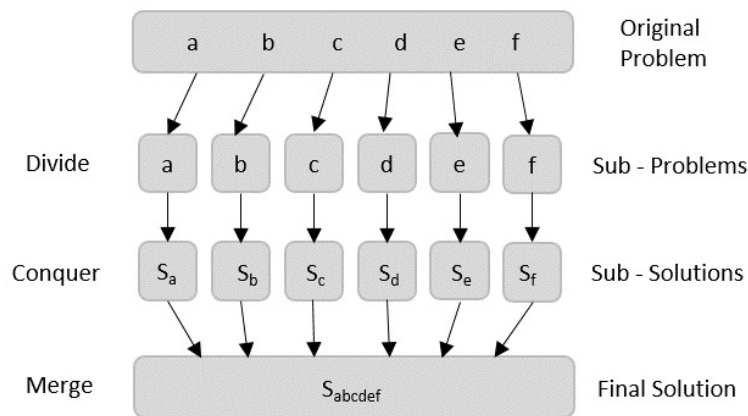


Figure 5: Divide and Conquer Algorithm [5]

Konsep Divide and Conquer sendiri bisa diambil langsung dari makna terjemahannya sendiri. Divide dapat dimaknai sebagai pembagian persoalan menjadi beberapa upa-persoalan yang memiliki kemiripan dengan persoalan semula namun berukuran lebih kecil dari sebelumnya, idealnya pembagian dilakukan secara merata atau hampir sama. Sementara itu, Conquer berarti menyelesaikan masing-masing persoalan. Dengan menggabungkan kedua teorema, Divide and Conquer merupakan algoritma yang menggabungkan solusi dari masing-masing upa-persoalan menjadi sebuah solusi utama dari persoalan semula, di mana pembagian dan penyelesaian dilakukan secara rekursif.

Algoritma ini serupa dengan algoritma Decrease and Conquer, hanya saja Divide and Conquer lebih menekankan optimalisasi kompleksitas dibanding pengurangan abstraksi persoalan.

2.4 Bounding Box Collision Detection

Deteksi kolisi merupakan salah satu fundamental penting dalam grafik komputer, simulasi fisik, dan pengembangan *game*. Di balik semua implementasi yang telah dilakukan, terdapat beberapa perhitungan matematika yang tidak dapat dihindar.

2.4.1 Axis-Aligned Bounding Box

Axis-Aligned Bounding Box (AABB) merupakan bentuk AABB merupakan kotak yang sejajar dengan sumbu koordinat (x, y, z) sehingga perhitungan menjadi sangat mudah dan efisien. Namun, karena orientasinya selalu mengikuti sumbu, metode ini kurang fleksibel dalam merepresentasikan objek yang mengalami rotasi. Proses pengecekan dari metode perhitungan ini bisa dicek melalui dua AABB yang saling beririsan, dilakukan dengan membandingkan rentang nilai pada masing-masing sumbu yang ada. Interval kedua bounding box dicek pada setiap sumbu, dilihat apakah saling tumpang tindih.

2.4.2 Hubungan Octree dengan AABB

Dalam implementasi Octree, setiap node merepresentasikan suatu ruang dalam bentuk Axis-Aligned Bounding Box (AABB). AABB digunakan sebagai representasi ruang karena bentuknya sederhana dan sejajar dengan sumbu koordinat, sehingga memudahkan proses pembagian ruang menjadi delapan bagian pada setiap subdivisi.

Selain sebagai representasi ruang, AABB juga digunakan dalam proses pengecekan interseksi dengan geometri objek. Pada implementasi ini, pengecekan interseksi antara bounding box dan segitiga dilakukan menggunakan pendekatan Separating Axis Theorem (SAT), yaitu dengan menguji apakah terdapat sumbu pemisah antara kedua objek.

Jika tidak ditemukan sumbu pemisah, maka bounding box dianggap beririsan dengan segitiga, sehingga node perlu diproses lebih lanjut. Sebaliknya, jika terdapat sumbu pemisah, node dapat diabaikan (pruned). Dengan kombinasi AABB sebagai representasi ruang dan SAT sebagai metode pengecekan, proses voxelization dapat dilakukan secara efisien.

3 Pendekatan Solusi

Solusi voxelization dilakukan dengan memanfaatkan struktur data Octree untuk merepresentasikan pembagian ruang tiga dimensi secara hierarkis. Pendekatan yang digunakan adalah Divide and Conquer, di mana ruang kubus awal yang mencakup seluruh objek akan dibagi secara rekursif menjadi delapan bagian hingga mencapai kedalaman maksimum atau memenuhi kondisi tertentu yaitu tidak ada kondisi intersect ruang dengan unit mesh. Dengan pendekatan ini, ruang kosong yang tidak berinteraksi dengan objek dapat dieliminasi lebih awal melalui proses pruning, sehingga meningkatkan efisiensi dibandingkan metode brute-force.

4 Implementasi

4.1 Struktur Data

Struktur program dibangun berdasarkan tiga komponen utama, yaitu **Types**, **Mesh**, dan **Octree**. Ketiga komponen ini digunakan untuk merepresentasikan data geometri serta struktur yang diperlukan dalam proses voxelization.

4.1.1 Types

```
1 struct Vertex
2 {
3     float x, y, z;
4 };
5
6 struct Face
7 {
8     int v1, v2, v3;
9 };
10
11 struct BoundingBox
12 {
13     Vertex min, max;
14 };
```

Selengkapnya lihat file `types.hpp` pada repository (lihat Lampiran A.1).

Modul **Types** mendefinisikan struktur data dasar yang digunakan untuk merepresentasikan elemen geometri dalam program. Struktur **Vertex** digunakan untuk merepresentasikan titik dalam ruang tiga dimensi dengan koordinat (x, y, z) , yang menjadi elemen dasar dalam pembentukan mesh. Selanjutnya, struktur **Face** merepresentasikan sebuah segitiga yang dibentuk oleh tiga vertex melalui penyimpanan indeks ketiga vertex tersebut.

Untuk mendukung proses partisi ruang, digunakan struktur **BoundingBox** yang merepresentasikan batas ruang tiga dimensi dalam bentuk axis-aligned bounding box, yang didefinisikan oleh titik minimum dan maksimum. Struktur ini berperan

penting dalam proses pembagian ruang pada Octree serta dalam pengecekan interseksi terhadap mesh.

Ketiga struktur ini menjadi fondasi utama dalam proses voxelization, di mana **Vertex** dan **Face** merepresentasikan geometri objek, sedangkan **BoundingBox** digunakan untuk melakukan partisi ruang secara efisien.

4.1.2 Mesh

```
1 class Mesh
2 {
3 public:
4     std::vector<Vertex> vertices;
5     std::vector<Face> faces;
6
7     BoundingBox findBoundingBox() const;
8 };
```

*Selengkapnya lihat file **Mesh.cpp** pada repository (lihat Lampiran A.1).*

Kelas **Mesh** digunakan untuk merepresentasikan objek tiga dimensi yang dibaca dari file **.obj**. Struktur ini menyimpan kumpulan vertex dalam bentuk **std::vector<Vertex>** serta kumpulan face dalam bentuk **std::vector<Face>**, di mana setiap face merepresentasikan segitiga yang terbentuk dari tiga vertex.

Selain sebagai penyimpanan data geometri, kelas ini juga menyediakan fungsi **findBoundingBox()** yang digunakan untuk menghitung bounding box dari seluruh objek. Bounding box ini diperoleh dengan mencari nilai minimum dan maksimum pada setiap sumbu (x, y, z) dari seluruh vertex dalam mesh.

Bounding box yang dihasilkan berperan sebagai ruang awal (root) dalam konstruksi Octree. Dengan demikian, **Mesh** tidak hanya berfungsi sebagai representasi data objek, tetapi juga menjadi dasar dalam proses partisi ruang dan pengecekan interseksi pada algoritma voxelization.

4.1.3 Octree

```
1 struct OctreeStats
2 {
3     std::atomic<int> numVoxels = 0;
4     std::vector<std::atomic<int>> nodesFormed;
5     std::vector<std::atomic<int>> nodesPruned;
6
7     OctreeStats(int maxDepth): nodesFormed(maxDepth + 1),
8                               nodesPruned(maxDepth + 1)
9     {
10         for (int i = 0; i <= maxDepth; i++) {
11             nodesFormed[i].store(0, std::memory_order_relaxed);
12             nodesPruned[i].store(0, std::memory_order_relaxed);
13         }
14     };
15 }
```

```

15
16 class OctreeNode
17 {
18 public:
19     BoundingBox box;
20     OctreeNode *children[8];
21     std::vector<int> faceIndices;
22     int depth;
23     bool isLeaf = false;
24
25     OctreeNode(const BoundingBox &b, int d);
26     ~OctreeNode();
27
28     void build(const Mesh &mesh, int maxDepth, const std::vector<
        int> &surroundFaces, OctreeStats &stats, ThreadPool *
        threadPool);
29
30 private:
31     void subdivide();
32     bool isIntersectFace(const Mesh &mesh, int faceIndex) const;
33 };
34
35 class Octree
36 {
37 public:
38     OctreeNode *root;
39     int maxDepth;
40     OctreeStats stats;
41
42     Octree(int depth);
43     ~Octree();
44
45     void buildTree(const Mesh &mesh);
46 };

```

Selengkapnya lihat file `Octree.cpp` pada repository (lihat Lampiran A.1).

Kelas `Octree` digunakan untuk merepresentasikan pembagian ruang tiga dimensi secara hierarkis dalam proses voxelization. Struktur ini memiliki node utama (*root*) yang merepresentasikan bounding box dari seluruh objek, serta parameter `maxDepth` yang menentukan kedalaman maksimum pembagian ruang. Selain itu, digunakan struktur `OctreeStats` untuk mencatat statistik selama proses konstruksi, seperti jumlah voxel yang terbentuk, jumlah node yang dibuat, serta jumlah node yang dipangkas pada setiap kedalaman.

Setiap node direpresentasikan oleh kelas `OctreeNode`, yang menyimpan bounding box, kedalaman node, serta daftar indeks face yang berpotensi berinteraksi dengan ruang tersebut. Node juga memiliki hingga delapan child yang merepresentasikan hasil subdivisi ruang. Atribut `isLeaf` digunakan untuk menandai node pada kedalaman maksimum yang akan direpresentasikan sebagai voxel.

Proses konstruksi Octree dilakukan secara rekursif melalui fungsi `build`. Pada setiap node, dilakukan pengecekan apakah bounding box node berinteraksi dengan face pada mesh. Jika tidak terdapat interseksi, node akan dipangkas dan tidak

diproses lebih lanjut. Jika terdapat interseksi dan kedalaman maksimum belum tercapai, maka node akan dibagi menjadi delapan child melalui proses subdivisi. Jika kedalaman maksimum telah tercapai, node ditandai sebagai leaf dan dihitung sebagai voxel.

Untuk meningkatkan efisiensi, setiap node hanya memproses subset face yang relevan, sehingga mengurangi jumlah pengecekan interseksi yang tidak diperlukan. Dengan pendekatan ini, Octree mampu menghindari eksplorasi seluruh ruang secara menyeluruh, sehingga proses voxelization menjadi lebih efisien dibandingkan metode berbasis grid uniform.

4.2 Algoritma Voxelization

Algoritma voxelization pada program ini diimplementasikan menggunakan pendekatan Divide and Conquer dengan memanfaatkan struktur data Octree. Proses dilakukan dengan membagi ruang tiga dimensi secara rekursif dan hanya memproses bagian ruang yang mungkin interseksi dengan objek. Dengan demikian, eksplorasi ruang kosong dapat dihindari sehingga meningkatkan efisiensi.

4.2.1 Inisialisasi

```
1 static bool loadObjFromFile(const std::string &fileName, Mesh &
    outMesh);
2 // Membaca file .obj, mem-parse vertex (v) dan face segitiga (f),
3 // melakukan validasi format dan indeks, lalu mengisi Mesh.
4 // Return false jika file tidak valid atau terjadi error.
```

Selengkapnya lihat file `FileSystem.cpp` pada repository (lihat Lampiran A.1).

Proses dimulai dengan membaca file `.obj` dan membangun objek `Mesh` yang berisi kumpulan vertex dan face. Selanjutnya, dilakukan perhitungan bounding box global dari seluruh objek menggunakan fungsi `findBoundingBox()`. Bounding box ini digunakan sebagai root node pada Octree.

Setelah itu, struktur `Octree` diinisialisasi dengan kedalaman maksimum (`maxDepth`) yang telah ditentukan. Root node kemudian dibentuk berdasarkan bounding box global tersebut.

4.2.2 Divide and Conquer

```
1 void OctreeNode::build(const Mesh &mesh, int maxDepth, const std::
    vector<int> &surroundFaces, OctreeStats &stats, ThreadPool*
    threadPool)
2 {
3     stats.nodesFormed[depth].fetch_add(1, std::memory_order_relaxed
        );
4
5     for (int faceIdx : surroundFaces)
6     {
```

```

7         if (isIntersectFace(mesh, faceIdx))
8         {
9             faceIndices.push_back(faceIdx);
10        }
11    }
12
13    if (faceIndices.empty())
14    {
15        isLeaf = true;
16        stats.nodesPruned[depth].fetch_add(1, std::memory_order_relaxed);
17        return;
18    }
19
20    if (depth == maxDepth)
21    {
22        isLeaf = true;
23        stats.numVoxels.fetch_add(1, std::memory_order_relaxed);
24        return;
25    }
26
27    isLeaf = false;
28    subdivide();
29
30    if (depth < 2 && faceIndices.size() > 1000){
31        std::vector<std::future<void>> tasks;
32
33        for (int i = 0; i < 8; i++)
34        {
35            tasks.push_back(threadPool->enqueue([this, &mesh, &
36                stats, maxDepth, i, threadPool]() {
37                children[i]->build(mesh, maxDepth, this->
38                    faceIndices, stats, threadPool);
39            }));
40        }
41
42        for (auto &t : tasks)
43        {
44            t.get();
45        }
46    }
47    else
48    {
49        for (int i = 0; i < 8; i++)
50        {
51            children[i]->build(mesh, maxDepth, this->faceIndices,
52                stats, threadPool);
53        }
54    }
55
56    faceIndices.clear();
57 }

```

Selengkapnya lihat file `Octree.cpp` pada repository (lihat Lampiran A.1).

Proses utama dilakukan secara rekursif pada setiap node Octree. Untuk setiap node,

dilakukan langkah-langkah sebagai berikut:

Pertama, dilakukan pengecekan apakah bounding box node berinteraksi dengan face pada mesh. Jika tidak terdapat interseksi, maka node dianggap tidak relevan dan proses dihentikan (pruning).

Jika terdapat interseksi dan kedalaman node belum mencapai `maxDepth`, maka node akan dibagi menjadi delapan child melalui proses subdivisi ruang. Masing-masing child merepresentasikan satu bagian dari delapan partisi ruang, dan proses rekursif dilanjutkan pada setiap child.

Jika kedalaman maksimum telah tercapai dan node masih berinteraksi dengan mesh, maka node tersebut ditandai sebagai leaf dan direpresentasikan sebagai voxel.

4.2.3 Kondisi Berhenti

Proses rekursif pada Octree akan berhenti apabila salah satu dari kondisi berikut terpenuhi:

Node tidak memiliki interseksi dengan mesh, sehingga dapat dipangkas dan tidak perlu diproses lebih lanjut.

Node telah mencapai kedalaman maksimum (`maxDepth`), sehingga tidak dilakukan subdivisi lebih lanjut dan node ditetapkan sebagai voxel apabila masih berinteraksi dengan mesh.

4.2.4 Pengecekan Interseksi

```
1 bool isIntersectFace(const Mesh &mesh, int faceIndex) const;
2 // Mengecek apakah sebuah face (segitiga) beririsan dengan bounding
  box node
3 // menggunakan pendekatan triangle-AABB intersection (Separating
  Axis Theorem)
```

Selengkapnya lihat file `Octree.cpp` pada repository (lihat Lampiran A.1).

Pengecekan interseksi dilakukan antara bounding box pada setiap node dengan face pada mesh. Setiap face direpresentasikan sebagai segitiga, sehingga proses ini melibatkan pengecekan apakah segitiga tersebut berpotensi beririsan dengan bounding box node.

Hasil dari pengecekan ini digunakan untuk menentukan apakah node perlu diproses lebih lanjut atau dapat diabaikan. Dengan hanya mempertimbangkan face yang relevan, jumlah perhitungan dapat dikurangi secara signifikan.

4.2.5 Output

```
1 bool FileSystem::saveObj(OctreeNode *root, int maxDepth, const std
  ::string &fileName, int &outNumVertices, int &outNumFaces)
2 // Menghasilkan mesh voxel dari Octree, lalu menyimpannya ke file .
  obj.
```

```

3 // Menulis vertex (v) dan face (f), serta mengembalikan jumlahnya.
4 // Return false jika gagal membuat file output.

```

Selengkapnya lihat file `FileSystem.cpp` pada repository (lihat Lampiran A.1).

Setelah proses konstruksi Octree selesai, setiap node leaf pada kedalaman maksimum yang berstatus terisi akan direpresentasikan sebagai voxel berbentuk kubus. Setiap voxel kemudian dikonversi menjadi representasi `.obj` dengan membentuk vertex dan face yang sesuai.

Selain menghasilkan file `.obj`, program juga menghitung dan menampilkan berbagai statistik, seperti jumlah voxel yang terbentuk, jumlah node pada setiap kedalaman, jumlah node yang dipangkas, serta waktu eksekusi program.

4.3 Kompleksitas

Secara teoritis, jumlah node pada Octree dapat berkembang hingga $O(8^d)$ dengan d sebagai kedalaman maksimum. Namun, dalam praktiknya, penggunaan pruning memungkinkan sebagian besar ruang kosong dieliminasi lebih awal, sehingga jumlah node yang diproses menjadi jauh lebih sedikit.

5 Extras

5.1 Concurrency

```

1 template <class F>
2 auto enqueue(F&& f) -> future<typename invoke_result<F>::type>
3 {
4     using ReturnType = typename invoke_result<F>::type;
5
6     auto taskPtr = make_shared<packaged_task<ReturnType> >>(forward
7         <F>(f));
8     future<ReturnType> result = taskPtr->get_future();
9
10    {
11        lock_guard<mutex> lock(queueMutex);
12        tasks_.emplace([taskPtr]() {
13            (*taskPtr)();
14        });
15
16        condition_.notify_one();
17        return result;
18    }

```

Selengkapnya lihat file `ThreadPool.hpp` pada repository (lihat Lampiran A.1).

Untuk meningkatkan performa, diterapkan konsep concurrency dengan memanfaatkan teknik multi-threading melalui kelas `ThreadPool` untuk menjalankan beberapa task secara bersamaan.

Parallelisasi diterapkan pada proses konstruksi Octree, di mana setelah sebuah node dibagi menjadi delapan child, setiap child dapat diproses secara independen sebagai task terpisah.

Untuk menghindari overhead yang berlebihan, parallelisasi hanya dilakukan pada kondisi tertentu, yaitu ketika kedalaman node masih rendah ($depth < 2$) dan jumlah face cukup besar (lebih dari 1000). Di luar kondisi tersebut, proses dijalankan secara sekuensial.

Pendekatan ini memungkinkan percepatan waktu eksekusi, terutama pada model dengan kompleksitas tinggi.

5.2 Object Viewer

```
1 // Pipeline 2D to 3D
2 ScreenPoint projectPoint(const Vertex& point, const Camera& cam,
3   const Viewport& viewport){
4     Vertex camPoint = worldToCamera(point, cam);
5
6     float fovRad = cam.fov * 3.1415926535f / 180.0f;
7     float focal = 1.0f / tan(fovRad * 0.5f);
8
9     float xScreen = (camPoint.x * focal) / camPoint.z;
10    float yScreen = (camPoint.y * focal) / camPoint.z;
11
12    return {
13        viewport.width * 0.5f + xScreen * viewport.width * 0.5f,
14        viewport.height * 0.5f - yScreen * viewport.height * 0.5f,
15        camPoint.z,
16        true
17    };
18 }
19 // Rendering dan Depth Sorting
20 sort(renderFaces.begin(), renderFaces.end(), [](const RenderFace& a
21   , const RenderFace& b){
22     return a.depth > b.depth;
23 });
24 // Back-face culling
25 if (CameraLogic::dot(normal, c1) >= 0) continue;
```

Selengkapnya lihat file *CameraLogic.cpp* dan *Viewer.cpp* pada repository (lihat Lampiran A.1).

Program dilengkapi dengan fitur visualisasi objek 3D untuk menampilkan hasil secara interaktif. Visualisasi ini dibangun menggunakan dua komponen utama, yaitu *CameraLogic* dan *Viewer*.

Komponen *CameraLogic* bertanggung jawab dalam perhitungan transformasi geometri, seperti menentukan posisi kamera, mengubah koordinat dari ruang dunia ke ruang kamera (*world-to-camera transformation*), serta memproyeksikan titik tiga dimensi menjadi koordinat dua dimensi pada layar menggunakan proyeksi perspek-

tif. Proses ini memanfaatkan parameter kamera seperti posisi, orientasi, dan field of view untuk menghasilkan tampilan yang sesuai.

Sementara itu, komponen **Viewer** bertanggung jawab dalam proses rendering dan interaksi pengguna. Visualisasi dilakukan menggunakan pustaka SFML, di mana objek ditampilkan dalam bentuk segitiga yang disusun dari face pada mesh. Untuk meningkatkan kualitas tampilan, dilakukan proses *back-face culling* untuk menghilangkan sisi yang tidak terlihat serta pengurutan berdasarkan kedalaman (*depth sorting*) agar objek dirender dengan urutan yang benar.

Selain itu, pengguna dapat berinteraksi dengan objek melalui input mouse dan keyboard, seperti melakukan rotasi kamera, pergeseran posisi, serta zoom in dan zoom out. Fitur ini memungkinkan pengguna untuk mengamati hasil voxelization dari berbagai sudut pandang secara lebih intuitif.

6 Uji Coba Program

Berikut ini merupakan beberapa contoh pengujian terhadap source code yang sudah dikembangkan oleh tim.

6.1 line.obj

Pengujian voxelization untuk file '.obj' hingga kedalaman 4.

```
PS C:\Users\jenni\OneDrive\Documents\Test\Tuc112_13524053_13524110\bin> ./voxelizer pumpkin.obj 4
Membaca file line.obj...

Output:
Banyak voxel yang terbentuk: 250
Banyak vertex yang terbentuk: 2000
Banyak faces yang terbentuk: 3000

Statistik node octree yang terbentuk:
1 : 8
2 : 48
3 : 168
4 : 560

Statistik node yang tidak perlu ditelusuri:
1 : 2
2 : 27
3 : 98
4 : 310

Kedalaman octree: 4
Lama waktu program berjalan: 11.1369 ms
Path: D:\Program Files\SFML\bin\test\output\voxelized_line.obj
```

Figure 6: Output CLI untuk file line.obj

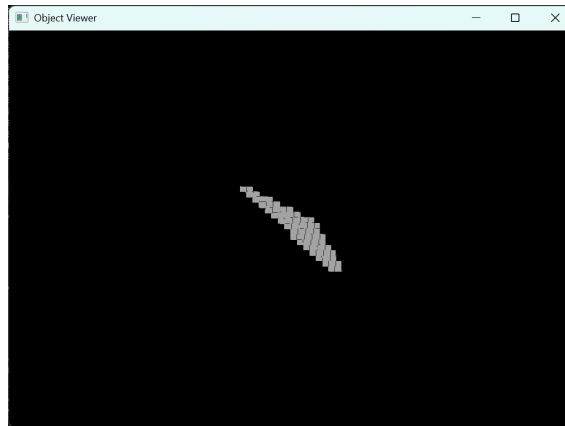


Figure 7: Output 3D untuk file voxelized_line.obj

6.2 teapot.obj

Pengujian voxelization untuk file '.obj' hingga kedalaman 5.

Catatan: Output menghasilkan ukuran file yang sangat besar, sehingga pada repositori hanya disertakan hasil pada kedalaman 4.

```
PS C:\Users\jenni\OneDrive\Documents\Test\Tuc112_13524053_13524110\bin> ./voxelizer teapot.obj 5

Output:
Banyak voxel yang terbentuk: 4001
Banyak vertex yang terbentuk: 32008
Banyak faces yang terbentuk: 48012

Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 416
4 : 1776
5 : 7504

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 12
3 : 194
4 : 838
5 : 3503

Kedalaman octree: 5
Lama waktu program berjalan: 69.7751 ms
Path dimana file .obj disimpan: test/output/voxelized_teapot.obj
```

Figure 8: Output CLI untuk file teapot.obj

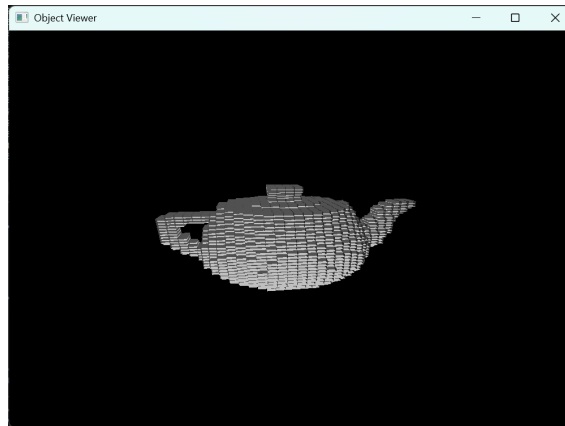


Figure 9: Output 3D untuk file voxelized_teapot.obj

6.3 cow.obj with maxDepth 6

Pengujian voxelization untuk file '.obj' hingga kedalaman 6.

Catatan: Output menghasilkan ukuran file yang sangat besar, sehingga pada repositori hanya disertakan hasil pada kedalaman 4.

```
PS C:\Users\jenni\OneDrive\Documents\Test\Tucil2_13524053_13524110\bin> ./voxelizer cow.obj 6
Banyak voxel yang terbentuk: 16410
Banyak vertex yang terbentuk: 131280
Banyak faces yang terbentuk: 196920

Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 384
4 : 1888
5 : 7616
6 : 31888

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 16
3 : 148
4 : 936
5 : 3630
6 : 15478

Kedalaman octree: 6
Lama waktu program berjalan: 236.577 ms
Path dimana file .obj disimpan: test/output/voxelized_cow.obj
```

Figure 10: Output CLI untuk file cow.obj dengan kedalaman 6

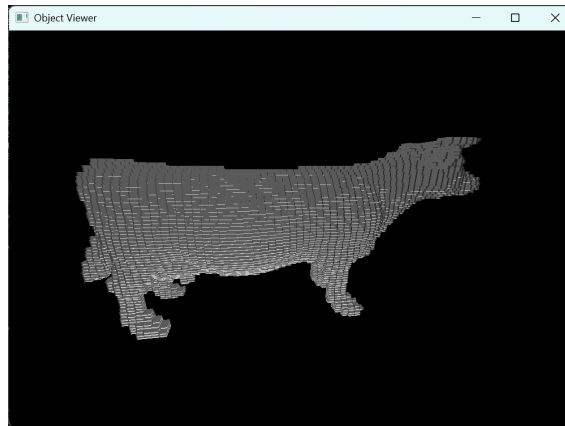


Figure 11: Output 3D untuk file voxelized_cow_6.obj

6.4 pumpkin.obj

Pengujian voxelization untuk file '.obj' hingga kedalaman 7.

Catatan: Output menghasilkan ukuran file yang sangat besar, sehingga pada repositori hanya disertakan hasil pada kedalaman 4.

```
PS C:\Users\jenni\OneDrive\Documents\Test\Tucil2_13524053_13524110\bin> ./voxelizer pumpkin.obj 7
Banyak faces yang terbentuk: 1007640

Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 448
4 : 2296
5 : 9912
6 : 41264
7 : 167144

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 8
3 : 161
4 : 1057
5 : 4754
6 : 20371
7 : 83174

Kedalaman octree: 7
Lama waktu program berjalan: 1756.92 ms
Path dimana file .obj disimpan: test/output/voxelized_pumpkin.obj
PS C:\Users\jenni\OneDrive\Documents\Test\Tucil2_13524053_13524110\bin>
```

Figure 12: Output CLI untuk file pumpkin.obj

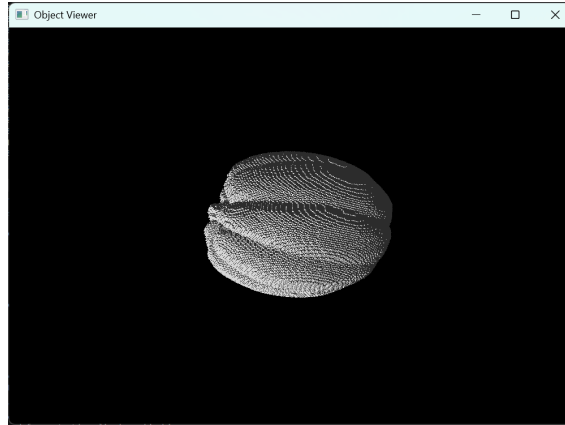


Figure 13: Output 3D untuk file voxelized_pumpkin.obj

6.5 cow.obj with maxDepth 8

Pengujian voxelization untuk file '.obj' hingga kedalaman 8.

Catatan: Output menghasilkan ukuran file yang sangat besar, sehingga pada repositori hanya disertakan hasil pada kedalaman 4.

```
PS C:\Users\jenni\OneDrive\Documents\Test\Tucil2_13524053_13524110\bin> ./voxelizer cow.obj 8
Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 384
4 : 1888
5 : 7616
6 : 31888
7 : 131280
8 : 531840

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 16
3 : 148
4 : 936
5 : 3630
6 : 15478
7 : 64800
8 : 264324

Kedalaman octree: 8
Lama waktu program berjalan: 4457.56 ms
Path dimana file .obj disimpan: test/output/voxelized_cow.obj
```

Figure 14: Output CLI untuk file cow.obj dengan kedalaman 8

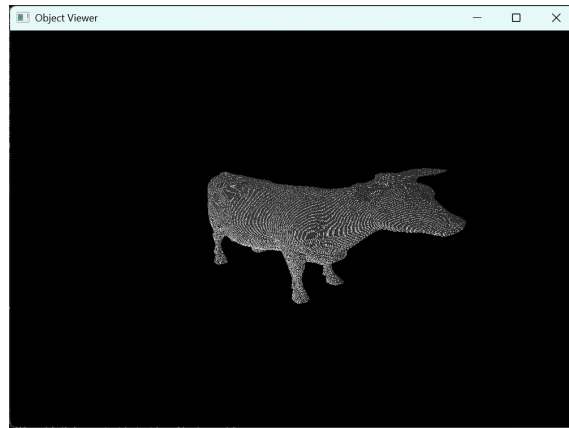


Figure 15: Output 3D untuk file voxelized_cow_8.obj

References

- [1] Castle Game Engine. How octree works, n.d. Diakses: 23 Maret 2026. URL: https://castle-engine.io/vrml_engine_doc/output/xsl/html/section.how_octree_works.html.
- [2] Geidav. Advanced octrees 1: Preliminaries, insertion strategies and max tree depth, 2014. Diakses: 23 Maret 2026. URL: <https://geidav.wordpress.com/2014/07/18/advanced-octrees-1-preliminaries-insertion-strategies-and-max-tree-depth/>.
- [3] Open3D. Voxelization. Diakses: 23 Maret 2026. URL: <https://www.open3d.org/docs/release/tutorial/geometry/voxelization.html>.
- [4] ResearchGate. Octants, octree structure, and linear octree illustration, n.d. Diakses: 23 Maret 2026. URL: https://www.researchgate.net/figure/a-Octants-b-Octree-Structure-c-Linear-Octree-Each-of-the-cubes-has-a-locational-code_fig1_221115507.
- [5] TutorialsPoint. Divide and conquer, n.d. Diakses: 27 Maret 2026. URL: https://www.tutorialspoint.com/data_structures_algorithms/divide_and_conquer.htm.
- [6] Zhen Zhang et al. A survey of voxelization methods in 3d reconstruction. *Sensors*, 22(18):7040, 2022. Diakses: 23 Maret 2026. URL: <https://www.mdpi.com/1424-8220/22/18/7040>, doi:10.3390/s22187040.

A Lampiran

A.1 Pranala Ke Repositori

https://github.com/mhps-null/Tucil2_13524053_13524110

A.2 Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Muhammad Haris Putra Sulastianto



Jennifer Khang

A.3 Tabel Spesifikasi Tugas Kecil

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program menampilkan informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj dan menghasilkan output	✓	
6	[Bonus] Program menggunakan concurrency	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj	✓	